

Chuletario Competitivo

Funciones, patrones y complejidades para resolver rápido y sin TLE. Cada bloque trae el código listo para copiar, cómo funciona por dentro, y qué cuesta en tiempo y memoria. Los ejemplos salen de los problemas resueltos en [TAP 2023–2025](#) e [ICPC Latin American](#).

ÍNDICE

BASE	ESTRUCTURAS
1 Entrada y salida	4 Listas y ordenamiento
2 Qué complejidad te entra	5 Dict, set y Counter
3 Manejo básico	6 Heap y deque
TÉCNICAS	MATEMÁTICA
7 Sumas acumuladas	11 Aritmética
8 Búsqueda binaria	12 Divisores y primos
9 Dos punteros	13 Operaciones con bits
10 Greedy y barridos	14 Geometría
AVANZADO	CIERRE
15 Strings	19 Combatir el TLE
16 Grafos	20 Trampas frecuentes
17 Programación dinámica	21 Plantilla base
18 Union-Find (DSU)	

La regla de oro. `input()` nativo es *lento*. Con más de ~10.000 líneas ya te cuesta segundos. En tu `A - Alfajores.py` el TLE en el test 28 fue exactamente esto. Usá siempre `sys.stdin.buffer.read()`.

sys.stdin.buffer.read().split()

Lee TODA la entrada de una sola vez

```
# La forma más rápida que existe en Python
import sys
data = sys.stdin.buffer.read().split()

n = int(data[0])
m = int(data[1])
v = list(map(int, data[2:2+n]))
o = list(map(int, data[2+n:2+n+m]))
```

Cómo funciona: `.buffer` lee bytes crudos sin decodificar a texto (ahorra ~30%). `.read()` trae todo el stdin en una sola llamada al sistema operativo en lugar de una por línea. `.split()` parte por cualquier espacio o salto de línea, así que **no importa cómo estén distribuidos los números en las líneas**.

Tiempo $O(T)$ sobre el total de bytes

Memoria $O(T)$

Usado en: **A - Alfajores**, **K - Kings Conquest**

Iterador con map + next

Cuando el formato es irregular

```
import sys
def main():
    data = sys.stdin.buffer.read().split()
    it = map(int, data) # lazy

    n = next(it) # 1ra fila
    w, l, tx, ty = (next(it) for _ in range(4))

    for _ in range(n): # pares
        x, y = next(it), next(it)

main()
```

Cómo funciona: `map` es perezoso: no convierte todo a int de golpe, sino a medida que pedís con `next()`. Ideal cuando la entrada mezcla formatos (una fila con 1 número, otra con 4, después pares).

$O(1)$ amortizado por valor

Memoria: no duplica la lista de ints

Ideal para: **F - Cold day at the beach**

Salida acumulada

Un solo write al final

```
import sys
salida = []
for ...:
    salida.append(str(resultado))

# separado por espacios
sys.stdout.write(' '.join(salida))

# uno por línea
sys.stdout.write('\n'.join(salida) + '\n')
```

Cómo funciona: cada `print()` hace un flush al sistema operativo. Con 10^5 prints eso son 10^5 syscalls. Acumular en lista y hacer **un solo write** convierte eso en 1 syscall.

$O(n)$ total

print() en loop: 10-50x más lento

Usado en: **A - Alfajores**, **I - Inés y sus compitas**

Formatos de salida

Los que más aparecen

```
lista = [3, 0, 5]

print(' '.join(map(str, lista))) # 3 0 5
print(*lista) # 3 0 5 (igual)
print(*lista, sep='\n') # uno por línea

# decimales: 6 cifras
print(f"{dist:.6f}")

# CUIDADO: print(a, " ", b) mete espacios dobles
print("A", cant) # correcto: A 3
```

Cómo funciona: `' '.join` necesita strings, por eso el `map(str, ...)`. `print(*lista)` desempaqueta la lista como argumentos y `print` ya separa con espacio por defecto — hace lo mismo con menos código.

$O(n)$

Ojo en: **F - Cold day** imprimía `print("A", " ", a) → "A 3"`

2 • Qué complejidad te entra

Mirá `n` en el enunciado *antes* de escribir código. Esta tabla te dice qué algoritmo podés permitirte con un límite típico de 1-2 segundos.

N HASTA	COMPLEJIDAD VIABLE	QUÉ USAR	VEREDICTO
10	$O(n!)$ / $O(2^n \cdot n)$	fuerza bruta, permutaciones	libre
20-25	$O(2^n)$	subconjuntos con bitmask, meet-in-the-middle	ok
100	$O(n^3)$	Floyd-Warshall, DP de intervalos	ok
1.000	$O(n^2)$	DP 2D, doble for	ok
5.000	$O(n^2)$	doble for — al límite en Python	justo

N HASTA	COMPLEJIDAD VIABLE	QUÉ USAR	VEREDICTO
10 ⁵	O(n log n)	sort, bisect, heap, set	el caso típico
10 ⁶	O(n)	un solo barrido, prefix sums, contadores	ok
10 ⁷	O(n)	solo con builtins en C (sum , Counter)	al límite
10 ⁹ +	O(log n) / O(1)	fórmula cerrada, binaria sobre la respuesta	matemática

■ **Regla práctica en Python:** si tu `n` es 10⁵ y estás escribiendo un `for` dentro de otro `for`, ya perdiste. Buscá cómo bajar a `O(n log n)`: ordenar, bisect, prefix sums, o un dict.

3 • Manejo básico

Lo que usás en todos los problemas fáciles

map, min, max, sum

Builtins que corren en C — siempre preferilos

```
a, b, c = map(int, input().split())

mayor = max(a, b, c)
menor = min(a, b, c)
medio = a + b + c - mayor - menor # truco

total = sum(lista)
mx = max(lista)
mn = min(lista)

# max/min con clave
p = max(puntos, key=lambda p: p[1])
```

Cómo funciona: `map(int, ...)` aplica `int` a cada string; el desempaqueado asigna cada valor a una variable — **falla si no hay exactamente 3 valores**. El truco del medio: suma total menos los extremos, sin ordenar.

`min/max/sum: O(n)` en C, ~5× más rápido que un `for`

Usado en: **D** - Duo, **N** - New Dimensions, **L** - Game series

Índices y rebanadas

Slicing sin salirte del rango

```
lista[-1] # último
lista[-2] # anteuúltimo
lista[:k] # primeros k
lista[k:] # desde k en adelante
lista[::-1] # invertida (copia)

# slicing NO tira error si te pasás
s = "hola"
s[2:99] # 'la' - no explota

lista.pop() # saca el último, O(1)
lista.pop(1) # saca índice 1, O(n)
```

Cómo funciona: el slicing recorta silenciosamente al límite. Por eso en **J** - Judgmental Crowd podés escribir `lista[i:i+5] == "boooo"` sin verificar el borde: cerca del final devuelve un string más corto que simplemente no coincide.

`slice: O(k)` — copia | `pop(0): O(n)` → usá deque

Usado en: **J** - Judgmental Crowd, **B** - Black and white, **H** - Hidden divisor

Inicializar acumuladores

Infinito y arreglos en cero

```
minimo = float('inf')
maximo = float('-inf')

INF = 1 << 62 # más rápido, entero puro

# arreglo de n ceros
s = [0] * n
s = [0] * (n + 1) # +1 para prefix sums

# matriz n x m - OJO con el *
g = [[0]*m for _ in range(n)] # bien
g = [[0]*m]*n # MAL: filas compartidas
```

Cómo funciona: `[0]*n` crea la lista en C, mucho más rápido que un `append` en loop. Pero `[fila]*n` copia la *referencia*: al modificar una fila cambian todas. Para matrices siempre usá la comprensión.

`[0]*n → O(n)` | `INF entero > float('inf')` en velocidad

Usado en: **L** - Games, **I** - Inés, **K** - Kings Conquest

enumerate, zip, range

Recorrer sin índices manuales

```
# índice + valor a la vez
for i, a in enumerate(lista):
    v = (a + i) % n

# dos listas en paralelo
for r, c in zip(filas, cols):
    ...

# rango hacia atrás
for i in range(n-1, -1, -1):
    ...

# de a pares sobre lista plana
filas = nums[0::2] # pares
cols = nums[1::2] # impares
```

Cómo funciona: `enumerate` y `zip` son iteradores en C — más rápidos que `for i in range(len(x))` con indexado manual, porque evitan una operación de índice por vuelta.

`O(n)`, sin copia extra | `nums[0::2]` si copia: `O(n)`

Usado en: **C** - Circularly, **K** - Kings Conquest

sort() vs sorted()

In-place o copia nueva

```

lista.sort()           # in-place, devuelve None
nueva = sorted(lista)  # copia ordenada

lista.sort(reverse=True)  # descendente

# ordenar por un campo de la tupla
puntos.sort(key=lambda p: p[0])  # por x
puntos.sort(key=lambda p: p[1])  # por y

# por dos criterios: x asc, y desc
puntos.sort(key=lambda p: (p[0], -p[1]))

```

Cómo funciona: Timsort es estable — los empates conservan su orden original. Eso te deja **ordenar por criterios encadenados**: ordená primero por el criterio secundario y después por el principal. `sort()` no devuelve nada, así que `x = lista.sort()` te deja `None`.

`O(n log n)` `sort(): memoria O(1) extra`

`sorted(): memoria O(n)`

Usado en: **C** - Chromatic, **H** - Hidden divisor, **F** - Cold day

Ordenar tuplas sin key

Python compara elemento por elemento

```

pares = [(3, 1), (1, 5), (1, 2)]
pares.sort()
# [(1, 2), (1, 5), (3, 1)]
# compara [0]; si empata, compara [1]

# truco: armá la tupla con lo que querés
umbrales = sorted((dh - dv, dv, dh)
                  for dv, dh in pares)

```

Cómo funciona: ordenar tuplas sin `key` es **más rápido** que con `lambda`, porque la comparación ocurre entera en C sin llamar a Python. Si podés construir la tupla en el orden que necesitás, hacelo.

`O(n log n)`, ~2× más rápido que con `key=lambda`

Usado en: **K** - Kings Conquest

count, index, in

Búsqueda lineal — cuidado en loops

```

lista.count(1)          # cuántos unos
lista.index(x)          # 1ra posición (ValueError si falta)
x in lista              # O(n) en lista
x in conjunto           # O(1) en set ← usá esto

# count sobre un subrango
lista[l:r].count(1)     # O(r-l) + copia

# en strings
texto.count('T')        # O(n), en C, rapidísimo

```

Cómo funciona: los tres recorren la lista entera. Están implementados en C así que para *una* llamada son rápidos, pero `count` dentro de un `for` es `O(n²)`. Si consultás muchas veces, pasá a `set` o a prefix sums.

1 llamada: `O(n)` en C dentro de un `for`: `O(n²)`

Usado en: **G** - Garlands, **I** - Inés

Comprensión de listasMás rápido que `append` en `loop`

```

A = [x - 1 for x in lista]          # 0-based
g = [-e for e in f]                 # negados
pares = [x for x in lista if x % 2 == 0] # filtro

# de strings a ints
v = [int(x) for x in datos[2:2+n]]
v = list(map(int, datos[2:2+n]))    # más rápido aún

# generador: no construye la lista
total = sum(x*x for x in lista)

```

Cómo funciona: la comprensión evita la búsqueda del método `.append` en cada vuelta (~30% más rápida). `map(int, ...)` gana a la comprensión porque no ejecuta bytecode de Python por elemento. Un **generador** (paréntesis en vez de corchetes) no reserva memoria para la lista completa.

`map`: el más rápido generador: memoria `O(1)`

Usado en: **A** - Alfajores, **C** - Circularly

set — pertenencia instantánea

Convertí una lista y consultá gratis

```
divisor_set = set(lista)      # O(n)

if x in divisor_set:         # O(1) ← clave
    ...

# operaciones de conjuntos
a & b      # intersección
a | b      # unión
a - b      # diferencia
len(set(lista)) # cuántos distintos
```

Cómo funciona: el set es una tabla de hash. Buscar no recorre nada: calcula el hash y salta directo a la posición. Convertir una lista a set cuesta $O(n)$ una sola vez y después cada consulta es constante — eso transforma un $O(n^2)$ en $O(n)$.

in: $O(1)$ promedio { Memoria $\sim 4\times$ la de una lista }

Usado en: **H - Hidden divisor** (buscar el divisor faltante)

dict con .get(clave, default)

Sin KeyError, sin if previo

```
d = {}
g = d.get          # cachear el método = más rápido

if dl < g(du, INF): # INF si no existe
    d[du] = dl

# contar ocurrencias
cnt = {}
for x in lista:
    cnt[x] = cnt.get(x, 0) + 1

# recorrer ordenado por clave
for k in sorted(d):
    ...
```

Cómo funciona: `.get(k, default)` devuelve el default si la clave falta, sin lanzar excepción. Guardar el método en una variable (`g = d.get`) evita la búsqueda del atributo en cada vuelta — un truco real dentro de loops calientes.

get/set: $O(1)$ promedio { **sorted(d):** $O(k \log k)$ }

Usado en: **K - Kings Conquest** (4 dicts de esquinas)

Counter

Contar todo de una

```
from collections import Counter

c = Counter(lista)
c[x]          # cuántas veces (0 si falta)
c.most_common(1) # [(valor, cantidad)]
c.most_common()  # todos, de mayor a menor

# también sobre strings
c = Counter("TPAUTP")
min(c['T'], c['P'], c['A'] + c['U'])
```

Cómo funciona: es un dict que devuelve `0` en vez de fallar con claves ausentes. El conteo interno está en C, así que `Counter(lista)` es más rápido que un loop manual con `.get`.

Construcción $O(n)$ { **most_common():** $O(k \log k)$ }

Alternativa limpia para: **G - Garlands**

defaultdict

Para listas de adyacencia y agrupaciones

```
from collections import defaultdict

# grafo
g = defaultdict(list)
g[u].append(v) # sin inicializar g[u]

# agrupar por clave
por_x = defaultdict(list)
for x, y in puntos:
    por_x[x].append(y)

# contador
cnt = defaultdict(int)
cnt[x] += 1
```

Cómo funciona: al pedir una clave que no existe, la crea llamando a la fábrica que le pasaste (`list` → `[]`, `int` → `0`). Te ahorra el `if k not in d` en cada inserción.

$O(1)$ por operación

El patrón de frecuencias. En **C - Circularly** el problema pedía contar pares que cumplen una condición. La solución $O(n^2)$ (comparar todos contra todos) era imposible; la $O(n)$ fue: *primer barrido* construye un arreglo de frecuencias de la clase `(A[i]+i) % n`, *segundo barrido* consulta esa frecuencia. Cuando veas "contar pares que cumplen X", pensá en frecuencias.

6 · Heap y deque

Cuando necesitás el mínimo, o insertar por los dos lados

heapq – cola de prioridad

Mínimo siempre a mano

```
import heapq

h = []
heapq.heappush(h, x)      # insertar
menor = heapq.heappop(h)  # sacar el mínimo
h[0]                      # mirar sin sacar, O(1)

heapq.heapify(lista)      # O(n), in-place

# MAX-heap: negá los valores
heapq.heappush(h, -x)
mayor = -heapq.heappop(h)

# con prioridad compuesta
heapq.heappush(h, (costo, nodo))
```

Cómo funciona: es un árbol binario guardado en una lista plana; el hijo de i está en $2i+1$ y $2i+2$. La raíz siempre es el mínimo. Solo hay **min-heap**: para máximos, negá al entrar y al salir.

push/pop: $O(\log n)$ peek h[0]: $O(1)$ heapify: $O(n)$

deque – cola por los dos extremos

Reemplaza a pop(0)

```
from collections import deque

q = deque()
q.append(x)      # al final, O(1)
q.appendleft(x)  # al inicio, O(1)
q.pop()          # del final, O(1)
q.popleft()      # del inicio, O(1) ← clave

q = deque(lista) # desde una lista
q.rotate(1)      # rota a la derecha
```

Cómo funciona: es una lista doblemente enlazada por bloques, así que insertar o sacar en cualquier extremo es constante. En una lista normal, `pop(0)` desplaza todos los elementos: $O(n)$. Con 10^5 operaciones eso es la diferencia entre 0,01 s y 10 s.

Ambos extremos: $O(1)$ Acceso al medio `q[i]`: $O(n)$

7 · Sumas acumuladas (prefix sums)

Muchas consultas de rango → $O(1)$ cada una

Prefix sum 1D

La técnica más rentable de todas

```
# n+1 posiciones: la 0 es "cero elementos"
pre = [0] * (n + 1)
for i in range(n):
    pre[i+1] = pre[i] + lista[i]

# suma de lista[l-1 .. r-1] (1-based)
suma = pre[r] - pre[l-1]

# suma de lista[i .. j] (0-based)
suma = pre[j+1] - pre[i]
```

Cómo funciona: `pre[i]` guarda el total de los primeros i elementos. La suma de un rango sale de restar dos acumulados: todo hasta r menos todo lo anterior a l . El $+1$ del tamaño evita tener que tratar $l = 0$ como caso especial.

Construcción: $O(n)$ Cada consulta: $O(1)$ Memoria: $O(n)$

Usado en: **L - Games** – 3 prefix sums en paralelo (pot2, imp, unos)

Varios prefix en paralelo

Un barrido, varias categorías

```
pot2 = [0] * (n + 1)
imp = [0] * (n + 1)
unos = [0] * (n + 1)

for i in range(n):
    x = lista[i]
    # cada posición copia lo acumulado
    pot2[i+1] = pot2[i]
    imp[i+1] = imp[i]
    unos[i+1] = unos[i]

    if x == 1:          unos[i+1] += 1
    elif (x & (x - 1)) == 0: pot2[i+1] += x
    elif x % 2 == 1:    imp[i+1] += x

# consulta del rango [l, r]
u = unos[r] - unos[l-1]
a = pot2[r] - pot2[l-1]
```

Cómo funciona: clasificás cada elemento en una categoría y mantenés un acumulado por categoría. Con q consultas pasás de $O(n \cdot q)$ a $O(n + q)$.

$O(n + q)$ total Sin esto: $O(n \cdot q)$ → TLE seguro

Usado en: **L - Games**

Mínimo/máximo acumulado

Prefijo y sufijo

```
# mínimo de los primeros i+1
pref = []
acum = float('inf')
for x in lista:
    if x < acum: acum = x
    pref.append(acum)

# mínimo desde i hasta el final
suf = [0] * (n + 1)
suf[n] = float('inf')
for i in range(n-1, -1, -1):
    suf[i] = min(suf[i+1], lista[i])
```

Cómo funciona: igual que la suma, pero el operador es `min`/`max`. Sirve cuando querés partir el arreglo en dos y necesitás lo mejor de cada lado. El sufijo se llena **hacia atrás**.

`O(n)` construcción, `O(1)` consulta

Usado en: **K - Kings Conquest** (pref_dh, suf_dh)

Diferencias (range update)

Sumar en muchos rangos, leer al final

```
dif = [0] * (n + 2)

# sumar v a todo el rango [l, r]
dif[l] += v
dif[r+1] -= v

# reconstruir el arreglo final
actual = 0
res = []
for i in range(n):
    actual += dif[i]
    res.append(actual)
```

Cómo funciona: el inverso del prefix sum. Marcás dónde empieza y dónde termina cada suma; al recorrer acumulando, el efecto se "enciende" en `l` y se "apaga" en `r+1`. Con `q` actualizaciones: `O(q + n)` en vez de `O(q · n)`.

Update: `O(1)`

Reconstrucción final: `O(n)`

8 · Búsqueda binaria

Requisito: la lista tiene que estar ordenada

bisect_left / bisect_right

Contar cuántos son menores

```
from bisect import bisect_left, bisect_right

B.sort() # O(m log m), obligatorio

for x in A: # O(n log m)
    menores = bisect_left(B, x) # cuántos < x
    menores_ig = bisect_right(B, x) # cuántos <= x
    iguales = bisect_right(B, x) - bisect_left(B, x)

# buscar solo desde cierta posición
j = bisect_left(B, x, desde)
```

Cómo funciona: `bisect_left` devuelve la posición donde *insertarías* `x` manteniendo el orden, a la izquierda de los iguales. Esa posición **es exactamente la cantidad de elementos estrictamente menores**. `bisect_right` inserta a la derecha, así que cuenta los `<= x`.

Cada búsqueda: `O(log n)` + sort inicial: `O(n log n)`

Usado en: **A - Alfajores**, **K - Kings Conquest**

Buscar en lista descendente

El truco de negar

```
# f está ordenada DESCENDENTE
# bisect solo funciona sobre ascendentes

g = [-e for e in f] # ahora es ascendente
# y conserva los MISMOs índices

# "f[j] <= r" equivale a "g[j] >= -r"
# y "primera posición con valor >= x" es bisect_left
j = bisect_left(g, -r, desde)
```

Cómo funciona: negar invierte el orden pero **no mueve los índices**, así que el índice que devuelve bisect sobre `g` sirve tal cual para indexar `f`. Es más barato que mantener dos listas o escribir tu propia binaria.

`O(log n)` por consulta

Usado en: **A - Alfajores**

insort – insertar ordenado

Mantener una lista siempre ordenada

```
from bisect import insert

insert(lista, x)    # inserta en su posición
```

Cómo funciona: encuentra la posición en $O(\log n)$, pero **insertar desplaza** todos los elementos siguientes: $O(n)$. Para pocas inserciones está bien; para muchas, acumulá todo y ordená una sola vez al final.

$O(n)$ por inserción (el desplazamiento manda)

n inserciones sueltas < acumular + sort

Binaria sobre la respuesta

"¿El mínimo valor tal que...?"

```
def alcanza(x):
    # True si x es suficiente
    ...

lo, hi = 0, 10**18
while lo < hi:
    mid = (lo + hi) // 2
    if alcanza(mid):
        hi = mid        # probá más chico
    else:
        lo = mid + 1    # hace falta más
print(lo)
```

Cómo funciona: aplica cuando la respuesta es **monótona**: si x alcanza, todo $> x$ también. En vez de probar cada valor, partís el rango a la mitad. Con $hi = 10^{18}$ son solo ~60 iteraciones.

$O(\log(\text{rango}) \cdot \text{costo de alcanza})$

9 • Dos punteros y ventana deslizante

Bajar $O(n^2)$ a $O(n)$ sin estructuras extra

Dos punteros desde los extremos

Sobre lista ordenada

```
lista.sort()
i, j = 0, len(lista) - 1

while i < j:
    s = lista[i] + lista[j]
    if s == objetivo:
        # encontrado
        i += 1; j -= 1
    elif s < objetivo:
        i += 1    # necesito más
    else:
        j -= 1    # necesito menos
```

Cómo funciona: como la lista está ordenada, si la suma es chica solo podés crecer moviendo i ; si es grande, solo podés achicar moviendo j . Cada puntero avanza a lo sumo n veces, así que el while total es $O(n)$.

$O(n)$ tras el sort { Total con sort: $O(n \log n)$ }

Enfoque correcto para: **J - Never Add Up to X**

Ventana deslizante

Subarreglo más largo que cumple X

```
izq = 0
suma = 0
mejor = 0

for der in range(n):
    suma += lista[der]    # entra

    while suma > limite:  # se pasó
        suma -= lista[izq]  # sale
        izq += 1

    largo = der - izq + 1
    if largo > mejor: mejor = largo
```

Cómo funciona: el puntero derecho avanza siempre; el izquierdo solo cuando la ventana deja de ser válida. Aunque hay un `while` dentro del `for`, **no es $O(n^2)$** : izq nunca retrocede, así que en total avanza n posiciones.

$O(n)$ amortizado { Memoria $O(1)$ }

Emparejar extremos

Uno ascendente contra otro descendente

```
r = sorted(map(int, input().split()))
a = sorted(map(int, input().split()), reverse=True)

sumas = [r[i] + a[i] for i in range(n)]
print(max(sumas) - min(sumas))
```

Cómo funciona: emparejar el más chico de una lista con el más grande de la otra **minimiza la dispersión** de las sumas. Es un intercambio clásico: si dos pares están "cruzados", intercambiarlos nunca empeora el resultado.

 $O(n \log n)$ por los dos sortsUsado en: **C - Chromatic****Filtro de mínimos por prefijo**

Descartar los elementos inútiles

```
f = []
minimo = float('inf')
for e in o:
    if e < minimo:      # mejora el récord
        f.append(e)
        minimo = e
# f queda ordenada estrictamente descendente
```

Cómo funciona: en un solo barrido te quedás solo con los elementos que mejoran el mínimo visto hasta ahora. Los demás son **dominados**: nunca van a ser la mejor opción, porque hay uno anterior más chico. Reduce drásticamente el tamaño del problema.

 $O(n)$, un solo barridoUsado en: **A - Alfajores**, **K - Kings Conquest** (puntos no dominados)**Bandera de estado**

Recorridos con memoria del paso anterior

```
f = 0
ficha = False

for j in range(n):
    if (fila[j:j+2] == "NN") and (not ficha):
        f += 1
        ficha = True      # ocupé esta posición
    else:
        ficha = False     # liberé
```

Cómo funciona: mantenés una variable booleana con lo que pasó en el paso anterior para evitar solapamientos. Es una máquina de estados mínima: **$O(n)$** y memoria **$O(1)$** .

 $O(n)$, memoria $O(1)$ Usado en: **B - Black and white**, **F - Fixture** (racha de ganados)**Umbral entre dos estrategias**

Elegir según el tamaño de la entrada

```
UMBRAL = 64

if total <= UMBRAL:
    # pocos elementos: barrido lineal directo,
    # sin overhead de llamar a bisect
    for a in v:
        for e in f: ...
else:
    # muchos: búsqueda binaria para saltar
    for a in v:
        j = bisect_left(g, -r, desde)
```

Cómo funciona: a veces el algoritmo asintóticamente peor gana en entradas chicas porque no paga el costo constante. Medís dónde se cruzan las curvas y elegís la rama en tiempo de ejecución. Técnica avanzada pero decisiva cuando estás al límite del TLE.

Best of both: $O(n \cdot k)$ o $O(n \log m)$ Usado en: **A - Alfajores** — así se resolvió el TLE del test 28

División, módulo y redondeo

Cuidado con los negativos

```
7 // 2      # 3
-7 // 2     # -4 ← redondea HACIA ABAJO
7 % 3       # 1
-7 % 3      # 2 ← el módulo es SIEMPRE positivo
```

```
# división hacia arriba (techo) sin floats
techo = -(-a // b)
techo = (a + b - 1) // b # solo si a,b > 0

divmod(7, 3) # (2, 1) – cociente y resto juntos
```

Cómo funciona: `//` redondea hacia el infinito negativo, no hacia cero. Eso hace que `%` en Python siempre devuelva el signo del divisor — muy cómodo para índices circulares (`(i-1) % n` nunca te da negativo).

`O(1)` para enteros chicos

Usado en: **G** - Going to the kiosk, **I** - Inés, **L** - Games

Índices circulares

Envolver un arreglo sobre sí mismo

```
siguiente = (i + 1) % n
anterior  = (i - 1) % n # funciona con i=0

# desplazar k posiciones
destino = (i + k) % n

# clasificar por clase de equivalencia
freq = [0] * n
for i, a in enumerate(A):
    freq[(a + i) % n] += 1
```

Cómo funciona: el módulo mapea cualquier índice al rango `[0, n)`. Como en Python el resto nunca es negativo, `(0-1) % n` da `n-1` directamente, sin el `if i == 0` que necesitarías en C++.

`O(1)`

Usado en: **C** - Circularly

Aritmética modular

Cuando pidan "módulo 10^9+7 "

```
MOD = 10**9 + 7

(a + b) % MOD
(a * b) % MOD

# potencia modular: O(log e), builtin en C
pow(a, e, MOD)

# inverso multiplicativo (MOD primo)
inv = pow(a, MOD - 2, MOD)

# división modular
(a * pow(b, MOD-2, MOD)) % MOD
```

Cómo funciona: `pow(a, e, m)` usa exponenciación binaria: eleva al cuadrado repetidamente en vez de multiplicar `e` veces. Nunca escribas `a**e % MOD` — eso calcula el número entero gigante primero y te cuelga.

`pow(a,e,m): O(log e)`

`a**e % m: puede ser O(enoorme)`

gcd, lcm, abs

Del módulo math

```
import math

math.gcd(a, b) # máximo común divisor, O(log min)
math.lcm(a, b) # mínimo común múltiplo (3.9+)
math.gcd(*lista) # gcd de toda una lista

# si no tenés math.lcm
lcm = a * b // math.gcd(a, b)

abs(x) # valor absoluto
```

Cómo funciona: `gcd` usa el algoritmo de Euclides: reemplaza `(a,b)` por `(b, a%b)` hasta que `b` es cero. Converge en `O(log min(a,b))`. En `lcm` dividí *antes* de multiplicar para no inflar el número.

`gcd: O(log min(a,b))`

isqrt y cuadrados perfectos

Sin errores de punto flotante

```
import math

math.isqrt(n)           # raíz entera EXACTA

# ¿es cuadrado perfecto?
r = math.isqrt(n)
if r * r == n:          # ← forma correcta
    ...

# forma frágil con floats
if math.sqrt(n) % 1 == 0: # falla con n grandes
    ...
```

Cómo funciona: `math.sqrt` devuelve un float de 64 bits: con `n` mayor a $\sim 2^{53}$ pierde precisión y te miente. `isqrt` trabaja con enteros puros y devuelve el mayor entero cuyo cuadrado no supera `n` — nunca se equivoca.

`isqrt:  $O(\log n)$ , exacto` `sqrt: impreciso para n grande`

Usado en: **H - Hidden divisor** — vale la pena cambiar el `sqrt` % 1 por `isqrt`

Todos los divisores de n

Solo hasta la raíz

```
def divisores(n):
    res = []
    i = 1
    while i * i <= n:
        if n % i == 0:
            res.append(i)
            if i != n // i:      # evita duplicar
                res.append(n // i)
            i += 1
    return sorted(res)
```

Cómo funciona: los divisores vienen en pares: si `i` divide a `n`, entonces `n//i` también. Uno de los dos siempre es $\leq \sqrt{n}$, así que recorriendo hasta ahí los encontrás todos. El `if i != n//i` evita contar dos veces la raíz cuando `n` es cuadrado perfecto.

`$O(\sqrt{n})$` Memoria `$O(d(n))$` — pocos divisores

Concepto central de: **H - Hidden divisor**

¿Es primo?

Test por divisiones hasta $\sqrt{n}$

```
def es_primo(n):
    if n < 2: return False
    if n < 4: return True
    if n % 2 == 0: return False
    i = 3
    while i * i <= n:
        if n % i == 0: return False
        i += 2      # solo impares
    return True
```

Cómo funciona: si `n` tiene un divisor mayor a $\sqrt{n}$, su pareja es menor, así que basta probar hasta ahí. Descartando los pares de entrada, recorrés la mitad de los candidatos.

`$O(\sqrt{n})$` Para muchas consultas: usá criba

Criba de Eratóstenes

Todos los primos hasta N de una vez

```
def criba(N):
    es = bytearray([1]) * (N + 1)
    es[0] = es[1] = 0
    for i in range(2, math.isqrt(N) + 1):
        if es[i]:
            # marca múltiplos desde i*i, en C
            es[i*i::i] = bytearray(len(es[i*i::i]))
    return [i for i in range(N+1) if es[i]]
```

Cómo funciona: marcás como compuesto cada múltiplo de cada primo. Arranca en `i*i` porque los menores ya fueron marcados por primos anteriores. El truco del `bytearray` con asignación por slice hace el marcado **entero en C**, mucho más rápido que un for.

`$O(N \log \log N)$` ≈ lineal Memoria: N bytes (no N enteros)

Factorización prima

$n = p_1^{a_1} \cdot p_2^{a_2} \cdot \dots$

```
def factorizar(n):
    f = {}
    d = 2
    while d * d <= n:
        while n % d == 0:
            f[d] = f.get(d, 0) + 1
            n //= d
        d += 1
    if n > 1:
        f[n] = f.get(n, 0) + 1 # primo grande
    return f
```

Cómo funciona: dividís por cada candidato hasta que ya no entra, lo que garantiza que solo encontrás primos. Si al terminar el bucle queda `n > 1`, ese resto es un primo mayor a $\sqrt{n}$ y va con exponente 1.

`$O(\sqrt{n})$` Cantidad de divisores: $\prod (a_i + 1)$

Comprobar si divide toda la lista

`all()` y `any()`

```
# ¿todos dividen al mayor?
if all(ult % c == 0 for c in lista):
    ...

# ¿alguno cumple?
if any(x > k for x in lista):
    ...
```

Cómo funciona: ambos cortan apenas saben la respuesta (short-circuit): `all` para en el primer `False`, `any` en el primer `True`. Con un generador dentro, no construyen ninguna lista intermedia.

`$O(n)$` peor caso, memoria `$O(1)$`

Reemplaza el loop con bandera de: **H (otras resolución)**

Los operadores

Y, O, XOR, desplazamientos

```

a & b      # AND: 1 donde AMBOS tienen 1
a | b      # OR:  1 donde ALGUNO tiene 1
a ^ b      # XOR: 1 donde son DISTINTOS
a << k     # multiplica por 2^k
a >> k     # divide por 2^k (hacia abajo)

```

```

1 << 62    # infinito entero práctico
x.bit_length() # bits necesarios
bin(x).count('1') # cuántos unos

```

Cómo funciona: operan sobre la representación binaria directamente.

`<< k` corre los bits a la izquierda agregando ceros, que es exactamente multiplicar por potencias de 2 — más rápido que la multiplicación.

0(1) para enteros de máquina

Usado en: **L - Games**, **K - Kings Conquest** (INF = `1 << 62`)

¿Es potencia de 2?

El truco de `x & (x-1)`

```

if x > 0 and (x & (x - 1)) == 0:
    # x es potencia de 2

# Por qué:
#   x   = 1000 (8)
#   x-1 = 0111 (7)
#   AND = 0000 → 0

# apagar el bit encendido más bajo
x & (x - 1)

# aislar el bit encendido más bajo
x & -x

```

Cómo funciona: una potencia de 2 tiene exactamente un bit en 1. Restar 1 apaga ese bit y enciende todos los de abajo, así que el AND siempre da 0. Cualquier otro número conserva al menos un bit en común.

0(1) — una instrucción

Ojo: x=0 pasa el test, filtralo

Usado en: **L - Games** para separar potencias de 2

Subconjuntos con bitmask

Para $n \leq 20$

```

for mask in range(1 << n):      # 2^n subconjuntos
    sub = []
    for i in range(n):
        if mask >> i & 1:      # ¿bit i encendido?
            sub.append(lista[i])
    # evaluar sub

```

Cómo funciona: cada número de 0 a 2^n-1 codifica un subconjunto: el bit `i` dice si el elemento `i` está incluido. Iterar sobre todos los enteros te da todos los subconjuntos, sin recursión.

0(2^n · n) — solo hasta n ≈ 20

Distancia entre puntos

hypot vs distancia al cuadrado

```
import math

# euclidiana
d = math.hypot(x2 - x1, y2 - y1)
d = math.dist((x1,y1), (x2,y2)) # 3.8+

# SOLO para comparar: evita la raíz
d2 = (x2-x1)**2 + (y2-y1)**2 # exacto, entero

# Manhattan
d = abs(x2-x1) + abs(y2-y1)
```

Cómo funciona: si solo necesitas *comparar* distancias, usá el cuadrado: la raíz es monótona, así que el orden no cambia. Ganás velocidad y —más importante— **trabajás con enteros exactos**, sin errores de punto flotante. `hypot` evita el desbordamiento intermedio de `x**2`.

`d²: 0(1) con enteros exactos`

`sqrt: float, error de redondeo`

Usado en: **F - Cold day at the beach** — ahí **convenía usar d²**

Bounding box

Rectángulo mínimo que contiene todo

```
xs = [p[0] for p in puntos]
ys = [p[1] for p in puntos]

xmin, xmax = min(xs), max(xs)
ymin, ymax = min(ys), max(ys)

ancho = xmax - xmin + 1 # celdas, no distancia
alto = ymax - ymin + 1

area = ancho * alto
perimetro = 2 * (ancho + alto)
```

Cómo funciona: un solo barrido con `min`/`max` sobre cada eje por separado. Ojo con el `+1`: si contás **celdas** de una grilla va, si medís **distancia** no. En **N - Nothofagus** el `+2` viene de sumar 1 celda de margen a cada lado.

`0(n)`

Usado en: **N - Nothofagus antarctica**, **K - Kings Conquest**

Producto cruz (orientación)

¿Giro a izquierda o a derecha?

```
def cruz(o, a, b):
    return ((a[0]-o[0]) * (b[1]-o[1]) -
            (a[1]-o[1]) * (b[0]-o[0]))

# > 0 → giro antihorario (izquierda)
# < 0 → giro horario (derecha)
# = 0 → colineales

# área del triángulo
area = abs(cruz(o, a, b)) / 2
```

Cómo funciona: es el determinante de los dos vectores. Su signo indica de qué lado de la recta `o→a` cae `b`. Con coordenadas enteras el resultado es entero **exacto**, sin ningún error de precisión — base de casi toda la geometría computacional.

`0(1), exacto con enteros`

Envolverte convexa (Andrew)

Convex hull

```
def hull(pts):
    pts = sorted(set(pts))
    if len(pts) <= 2: return pts

    def media(ps):
        h = []
        for p in ps:
            while len(h) >= 2 and cruz(h[-2], h[-1], p)
            <= 0:
                h.pop()
            h.append(p)
        return h

    return media(pts)[-1] + media(pts[:-1])[:-1]
```

Cómo funciona: ordenás por `x` y construís la cadena inferior y la superior. Mientras los últimos tres puntos no giren en el sentido correcto, sacás el del medio. Cada punto entra y sale a lo sumo una vez, así que las dos pasadas son `0(n)`; el costo lo domina el sort.

`0(n log n) por el sort`

Las pasadas: 0(n) amortizado

Métodos esenciales

Todos corren en C

```
s.strip()           # saca espacios/saltos de los bordes
s.split()           # por espacios
s.split(',')         # por un separador
s.count('T')         # ocurrencias,  $O(n)$ 
s.find('ab')         # índice o -1
s.replace('a','b')
s.upper() / s.lower()
s.startswith('ab') / s.endswith('ab')

sorted(s)           # lista de caracteres ordenada
''.join(sorted(s))  # string ordenado
```

Cómo funciona: `strip()` es casi obligatorio al leer con `input()`, porque el `\n` final se cuela y rompe las comparaciones. Todos estos métodos están en C, así que una llamada suelta sobre 10^6 caracteres es instantánea.

`count/find/replace:  $O(n)$`

Usado en: **G - Garlands**, **A - Artifact to print**

Buscar patrones por ventana

Slicing sin verificar bordes

```
for i in range(len(s)):
    if s[i:i+2] == "ha":    p += 1
    elif s[i:i+5] == "boooo": p -= 1
    elif s[i:i+5] == "bravo": p += 3

# contar sin solapamiento (más rápido)
s.count("ha")
```

Cómo funciona: el slicing recorta al final del string sin lanzar error, así que no necesitas el `if i+5 <= len(s)`. Cada rebanada crea un string nuevo de largo `k`, por eso el costo es $O(n \cdot k)$ — con `k` chico da lo mismo.

`$O(n \cdot k)$ ,  $k$  = largo del patrón` `s.count():  $O(n)$  en C`

Usado en: **J - Judgmental Crowd**, **B - Black and white**

Subsecuencia en orden

¿Aparecen las letras, aunque separadas?

```
objetivo = "TAP"
j = 0
for c in lista:
    if c == objetivo[j]:
        j += 1
        if j == len(objetivo):
            break

print("S" if j == len(objetivo) else "N")
```

Cómo funciona: un puntero greedy sobre el patrón. Cada vez que encontrás la letra que esperabas, avanzás. Es óptimo: tomar la primera aparición posible **nunca** te deja en peor situación que esperar una posterior.

`$O(n)$ , memoria  $O(1)$`

Usado en: **A - Artifact to print**

Construir strings sin $O(n^2)$

Acumula y uní al final

```
# MAL: cada += copia todo el string
s = ""
for x in lista:
    s += str(x)           #  $O(n^2)$ 

# BIEN
partes = []
for x in lista:
    partes.append(str(x))
s = ''.join(partes)      #  $O(n)$ 
```

Cómo funciona: los strings son inmutables, así que `s += x` crea uno nuevo copiando todo lo anterior. Con `n` concatenaciones eso es $O(n^2)$. `join` mide primero el total, reserva una vez y copia una vez.

`+= en loop:  $O(n^2)$` `join:  $O(n)$`

Lista de adyacencia

Cómo guardar el grafo

```
g = [[] for _ in range(n + 1)] # 1-based

for _ in range(m):
    u, v = next(it), next(it)
    g[u].append(v)
    g[v].append(u) # si NO es dirigido

# con peso
g[u].append((v, w))
```

Cómo funciona: una lista de listas donde `g[u]` son los vecinos de `u`. Ocupa $O(n + m)$, contra $O(n^2)$ de la matriz de adyacencia — indispensable con $n = 10^5$.

Memoria $O(n + m)$

Recorrer vecinos: $O(\text{grado})$

BFS — camino mínimo sin pesos

Por niveles, con deque

```
from collections import deque

dist = [-1] * (n + 1)
dist[origen] = 0
q = deque([origen])

while q:
    u = q.popleft()
    for v in g[u]:
        if dist[v] == -1: # no visitado
            dist[v] = dist[u] + 1
            q.append(v)
```

Cómo funciona: visita en orden de distancia creciente, así que la primera vez que llegas a un nodo ya es por el camino más corto. El `dist[v] == -1` hace de "visitado" y de distancia a la vez. **Solo sirve si todas las aristas pesan lo mismo.**

$O(n + m)$

Memoria $O(n)$

DFS iterativo

Sin recursión: evita el límite de pila

```
visto = [False] * (n + 1)
pila = [origen]

while pila:
    u = pila.pop()
    if visto[u]: continue
    visto[u] = True
    for v in g[u]:
        if not visto[v]:
            pila.append(v)
```

Cómo funciona: igual que el BFS pero con `pop()` del final en vez de `popleft()`. Python limita la recursión a ~1000 niveles: con $n = 10^5$ un DFS recursivo **revienta con RecursionError**. La versión iterativa no tiene ese techo.

$O(n + m)$

Recursivo: **RecursionError** con n grande

Dijkstra — con pesos positivos

BFS pero con heap

```
import heapq

INF = 1 << 62
dist = [INF] * (n + 1)
dist[origen] = 0
h = [(0, origen)]

while h:
    d, u = heapq.heappop(h)
    if d > dist[u]: continue # entrada vieja
    for v, w in g[u]:
        nd = d + w
        if nd < dist[v]:
            dist[v] = nd
            heapq.heappush(h, (nd, v))
```

Cómo funciona: el heap siempre te entrega el nodo pendiente más cercano, así que al sacarlo su distancia ya es definitiva. El `if d > dist[u]: continue` descarta entradas obsoletas — más barato que borrarlas del heap. **No funciona con pesos negativos.**

$O((n + m) \log n)$

Memoria $O(n + m)$

DP 1D

El esqueleto que sirve para casi todo

```
dp = [0] * (n + 1)
dp[0] = caso_base

for i in range(1, n + 1):
    dp[i] = max(dp[i-1],
               dp[i-2] + valor[i])

print(dp[n])
```

Cómo funciona: `dp[i]` guarda la mejor respuesta usando los primeros `i` elementos. Como cada estado se calcula una sola vez y se reutiliza, evitás el recálculo exponencial de la recursión ingenua.

`O(n · transiciones)``Memoria O(n)`**Mochila 0/1**

Con una sola fila

```
dp = [0] * (W + 1)

for peso, valor in objetos:
    # hacia ATRÁS: cada objeto una sola vez
    for w in range(W, peso - 1, -1):
        cand = dp[w - peso] + valor
        if cand > dp[w]:
            dp[w] = cand

print(dp[W])
```

Cómo funciona: el recorrido **hacia atrás** es lo esencial: garantiza que `dp[w-peso]` todavía no incluye el objeto actual. Si fueras hacia adelante estarías permitiendo usarlo varias veces (que es justo la mochila *ilimitada*).

`O(n · W)``Memoria O(W) – una fila`**lru_cache – memoización**

DP recursiva sin escribir la tabla

```
import sys
from functools import lru_cache
sys.setrecursionlimit(300000)

@lru_cache(maxsize=None)
def f(i, resto):
    if i == n: return 0
    mejor = f(i + 1, resto)
    if lista[i] <= resto:
        mejor = max(mejor, f(i+1, resto-lista[i]) +
                    v[i])
    return mejor
```

Cómo funciona: el decorador guarda en un dict el resultado de cada combinación de argumentos. Si volvéis a llamar con los mismos, devuelve el guardado sin recalcular. **Los argumentos deben ser hashables** — tuplas sí, listas no.

`O(estados · transición)``Memoria O(estados) – puede explotar``Recursión: subí el límite`**LIS – subsecuencia creciente más larga**Con bisect, en $O(n \log n)$

```
from bisect import bisect_left

cola = []
for x in lista:
    i = bisect_left(col, x)  # bisect_right si
    # permite <=
    if i == len(col):
        col.append(x)
    else:
        col[i] = x

print(len(col))  # SOLO el largo, no la subsecuencia
```

Cómo funciona: `cola[i]` guarda el **menor valor final posible** para una subsecuencia creciente de largo `i+1`. Mantener finales chicos deja más margen para extender después. El largo de `cola` es la respuesta, pero su contenido *no* es la subsecuencia real.

`O(n log n)``La versión DP simple es O(n²)`

Estructura completa

Con compresión de caminos y unión por tamaño

```
padre = list(range(n + 1))
tam   = [1] * (n + 1)

def find(x):
    # compresión iterativa (sin recursión)
    raiz = x
    while padre[raiz] != raiz:
        raiz = padre[raiz]
    while padre[x] != raiz:
        padre[x], x = raiz, padre[x]
    return raiz

def union(a, b):
    ra, rb = find(a), find(b)
    if ra == rb:
        return False
    if tam[ra] < tam[rb]:
        ra, rb = rb, ra
    padre[rb] = ra
    tam[ra] += tam[rb]
    return True

# cuántas componentes quedaron
componentes = sum(1 for i in range(1, n+1) if find(i) == i)
```

Cómo funciona: cada conjunto es un árbol y su raíz lo identifica. *Compresión de caminos:* al buscar, reengancha todos los nodos del camino directo a la raíz. *Unión por tamaño:* el árbol chico cuelga del grande, para que nunca se vuelva profundo. Juntas dejan cada operación en tiempo prácticamente constante.

Cuándo usarlo: conectividad con aristas que se van agregando, Kruskal para el árbol generador mínimo, o contar grupos de elementos relacionados.

find/union: $O(\alpha(n)) \approx O(1)$

Memoria $O(n)$

$\alpha(n) < 5$ para cualquier n real

EN VEZ DE	USÁ	POR QUÉ	GANANCIA
<code>input()</code> en loop	<code>sys.stdin.buffer.read().split()</code>	1 syscall en lugar de n	10–50×
<code>print()</code> en loop	acumular + <code>sys.stdout.write</code>	1 flush en lugar de n	10–50×
código suelto	todo dentro de <code>def main()</code>	las variables locales son más rápidas que las globales	15–30%
<code>while</code>	<code>for</code> / <code>range</code>	el for itera en C, sin evaluar condición en Python	20–40%
<code>for</code> + <code>append</code>	comprensión o <code>map</code>	evita buscar el método en cada vuelta	30%
<code>x in lista</code>	<code>x in set</code>	hash en vez de barrido lineal	$O(n) \rightarrow O(1)$
<code>lista.pop(0)</code>	<code>deque.popleft()</code>	no desplaza los elementos	$O(n) \rightarrow O(1)$
<code>s += x</code> en loop	<code>''.join(partes)</code>	no recopia el string cada vez	$O(n^2) \rightarrow O(n)$
<code>d.get(k)</code> en loop	<code>g = d.get</code> afuera	evita la búsqueda del atributo	10–20%
<code>math.sqrt(a)</code> para comparar	comparar los cuadrados	sin raíz y con enteros exactos	2–3× + precisión
<code>sort(key=lambda ...)</code>	ordenar tuplas directas	comparación entera en C	2×
consultas de rango en loop	prefix sums	precalcular una vez	$O(n \cdot q) \rightarrow O(n+q)$

Si ya optimizaste todo y sigue el TLE: el problema es el algoritmo, no el código. Volvé a mirar `n` en el enunciado y la tabla de la sección 2. Casi siempre hay una estructura (set, prefix, bisect, heap) que te baja un factor `n` entero.

Casos borde del enunciado

```
# n = 1: ¿el bucle siquiera corre?
if n == 1:
    print(...)
    return

# lista vacía → min()/max() explotan
if not lista: ...

# índice fuera de rango en i-1 / i+1
if i + 1 < n and lista[i] + lista[i+1] == x:
```

Qué revisar: el mínimo y el máximo de cada restricción del enunciado. En **K - Kings Conquest** el caso `n == 1` se resuelve antes de todo el algoritmo. En **H - Hidden divisor**, `n == 1` con el valor `1` es un caso completamente aparte.

Casos límite en: **H - Hidden divisor**, **K - Kings Conquest**

Off-by-one

```
# prefix sums: 1-based vs 0-based
suma = pre[r] - pre[l-1]    # si l,r son 1-based
suma = pre[r+1] - pre[l]    # si son 0-based

# celdas vs distancia
ancho = xmax - xmin + 1    # cantidad de celdas
ancho = xmax - xmin        # distancia

# pasar a 0-based al leer
A = [x - 1 for x in lista]
```

Qué revisar: definí desde el principio si trabajás 0-based o 1-based y no lo mezcles. Si el enunciado da índices 1-based, convertí al leer (`x - 1`) o reservá `n+1` posiciones y dejá la 0 sin usar.

Convención usada en: **C - Circularly**, **L - Games**

Enteros y flotantes

```
# Python no desborda enteros — no te preocupes
10**18 * 10**18    # funciona

# pero los floats SÍ pierden precisión
0.1 + 0.2 == 0.3    # False
math.sqrt(10**18) % 1 == 0    # poco confiable

# comparar floats con tolerancia
abs(a - b) < 1e-9

# mejor: quedate en enteros
math.isqrt(n) ** 2 == n
```

Qué revisar: si el problema es de enteros, resuelvo con enteros. El único lugar donde *tenés* que usar floats es cuando piden un resultado decimal — y ahí usá el formato exacto que pide el enunciado (`{d:.6f}`).

Riesgo presente en: **H - Hidden divisor** (`sqrt % 1`)

Bucles que no terminan

```
# un while True necesita SIEMPRE una salida
while True:
    ...
    if condicion: break    # ¿seguro que se cumple?

# si el estado puede repetirse, se cuelga
# → replanteá con for de iteraciones acotadas,
# o probá que cada vuelta reduce algo
```

Qué revisar: todo `while` necesita una **cantidad medible** que decrezca en cada vuelta. En **J - Never Add Up to X** el `while True` con intercambios puede volver a un estado ya visto y no terminar nunca — por eso da error en el juez aunque pase los ejemplos.

El bug de: **J - Never Add Up to X** (no anda)

template.py

Lectura rápida, main(), salida acumulada

```
import sys
from bisect import bisect_left, bisect_right
from collections import deque, defaultdict, Counter
import heapq, math

def main():
    data = sys.stdin.buffer.read().split()
    if not data:
        return
    it = map(int, data)

    # ---- lectura ----
    n = next(it)
    lista = [next(it) for _ in range(n)]

    # ---- solución ----
    salida = []
    for x in lista:
        salida.append(str(x))

    # ---- salida ----
    sys.stdout.write('\n'.join(salida) + '\n')

if __name__ == '__main__':
    main()
```

Por qué así: todo el trabajo pasa dentro de `main()`, donde las variables son **locales** — Python las resuelve por índice en un arreglo, no buscando en un diccionario global. Solo por eso ganás entre 15% y 30% sin tocar el algoritmo.

Los imports arriba no cuestan nada apreciable y te evitan tener que acordarte del nombre exacto bajo presión.

Locales vs globales: +15–30%**I/O: 1 read + 1 write****Orden de ataque para cada problema:**

- 1 · Leé `n` y las restricciones → mirá la tabla de la sección 2 para saber qué complejidad te entra.
- 2 · Resolvé primero el caso chico a mano, y buscá el patrón.
- 3 · Escribí la versión clara, aunque sea lenta.
- 4 · Si el `n` no da, buscá qué estructura elimina el factor de más: set, prefix, bisect, heap.
- 5 · Recién al final aplicá los trucos de la sección 19.